SELF LEARNING ACTIVITY REPORT

Submitted in partial fulfilment of the Requirements for
the Academics of 5th Semester, B.E in CSE (Cyber Security)

DEVOPS

25CYAEC59

In

Dept. of CSE (CYBER SECURITY)

By

Syed Shaibaan Hussain
USN: 1MS24CY056

Under the guidance of

Mr. Prashanth Reddy C
Senior Lecturer
Department of CSE (AI & ML) and CSE (Cyber Security)

M. S. Ramaiah Institute of Technology

(Autonomous Institute Affiliated to VTU)
MSR Nagar, MSRIT Post
Bangalore-560054, Karnataka, India

Certificate

This is to certify that the work carried out is a bonafide record of the project based learning activity work on the DevOps course done by Syed Shaibaan Hussain (1MS24CY056) under my supervision and guidance, in partial fulfilment of the requirements for the award of the Bachelor of Engineering in Computer Science & Engineering (Cyber Security) from Ramaiah Institute of Technology, Bangalore for the academic year 2026-2027.

Candidate Sign

Faculty Sign

LIST OF PROJECT BASED LEARNING COMPONENTS

COURSE NAME: DevOps **COURSE CODE:** 24CYAEC59
CREDITS: 1:0:0 **CONTACT HOURS:** 15L
MAX. MARKS: 20

Sl. No	Description	Marks
1	**CI/CD Pipeline using Git, Maven/Gradle and Jenkins.** Set up a version-controlled build pipeline for a sample application. Create a Git repository, use feature branching and merging, configure a Maven or Gradle build with dependency management, integrate the project into a Jenkins job that pulls from Source Code Management (SCM), triggers automatically on commit, executes the build, performs a post-build action, and add one CI-related enhancement (automated tests, build versioning, or GitHub Actions). *Covers: Unit 1 (DevOps culture/pipeline), Unit 2 (Git, Maven/Gradle), Unit 3 (Jenkins CI). Maps to CO1, CO2, CO3.*	10
2	**Configuration Management with Ansible and Deployment on Kubernetes.** Write an Ansible playbook (using variables, handlers, and at least one role) to configure and deploy the application built in Component 1. Containerize the application and write Kubernetes manifests (Deployment, Service, and a ConfigMap/PVC where applicable). Deploy on a local cluster (Minikube/Kind), demonstrate scaling using replica sets, show the use of labels and selectors, and add one deployment-related enhancement (rollback, ConfigMap, probes, multi-stage Dockerfile, second microservice, or health endpoint). *Covers: Unit 4 (Ansible), Unit 5 (Kubernetes). Maps to CO4, CO5.*	10

Assessment Rubrics

Evaluation for **each** 10-mark component will be based on the following rubrics.

Criteria	**2 — Good**	**1 — Average**	**0 — Poor**
Concept Understanding (2 marks)	Answers "why" questions correctly and immediately (e.g., webhook vs. polling, what a handler does, Deployment vs. ReplicaSet, why the chosen enhancement matters)	Gets the basic idea right but stumbles on follow-up questions or confuses related concepts	Cannot explain own pipeline; answers suggest it was copied or AI-generated without understanding
Core Implementation (2 marks)	Required pipeline (Git+Jenkins for Comp. 1 / Ansible+K8s+scaling for Comp. 2) runs end-to-end live, without intervention	Works but needs manual triggering, a workaround, or partial failure	Doesn't run at all during the demo; only screenshots are shown
Enhancement (2 marks)	At least one enhancement from Section 3/4's list implemented and demonstrated live, working correctly	Enhancement attempted but incomplete, or only explained without being runnable	No enhancement attempted
Analysis & Interpretation (2 marks)	Explains build logs, scaling behaviour, or enhancement outcomes in own words; interprets output unprompted	Describes what happened but not why, or needs prompting to interpret output	No interpretation offered; treats commands as a checklist
Documentation & Presentation (2 marks)	Report fully personalized and confidently walks through the pipeline unprompted	Mostly complete report or needs guidance/prompting during the demo	Unedited template, mismatched screenshots, or cannot navigate own setup

Marks Awarded

Sl. No	Concept Understanding (2)	Core Implementation (2)	Enhancement (2)	Analysis & Interpretation (2)	Documentation & Presentation (2)	Total
1						
2						
TOTAL MARKS						

Component 1: CI/CD Pipeline using Git, Maven and Jenkins

Abstract

The Unit Converter API converts Celsius to Fahrenheit and kilometres to miles, with a separate health route. This component stores the Java application on GitHub and builds it with Maven through Jenkins. Fifteen automated tests check the calculations and invalid requests before the executable JAR is packaged and archived.

Introduction

Continuous Integration gives quick feedback when a source change breaks a working application. Git records the changes and supports separate branches. Maven compiles Java, resolves dependencies and packages the application; Jenkins repeats those steps automatically after detecting a new commit.

The application is deliberately small: Java 17, Spring Boot 3.5.16, three GET routes, and no database or frontend. Its repository is `https://github.com/05Shaibaan/Devops-Project`. The same JAR is used in the deployment component.

Implementation

Development Environment

- Windows with Ubuntu 26.04.1 under WSL2; Git for Windows and Linux Git.
- Maven 3.9.12 and OpenJDK 21 in Ubuntu, compiling with the Java 17 release target.
- Spring Boot 3.5.16 with Spring Web and the Spring Boot test starter.
- Jenkins 2.580.2 with Pipeline, Git, JUnit and Timestamper plugins.

Application Code and Behaviour

The temperature route applies Fahrenheit = Celsius times 9/5 + 32. The distance route applies miles = kilometres times 0.621371. Each response includes the input value, both units and a result rounded to four decimal places. Inputs must be finite and within plus or minus one billion. Distance also rejects negative values. Spring handles missing and malformed query parameters with HTTP 400 responses.

`src/main/java/io/github/shaibaan/converter/ConversionController.java`

```
1  package io.github.shaibaan.converter;
2
3  import java.math.BigDecimal;
4  import java.math.RoundingMode;
5  import java.util.Map;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.web.bind.annotation.GetMapping;
8  import org.springframework.web.bind.annotation.RequestParam;
9  import org.springframework.web.bind.annotation.RestController;
10 import org.springframework.web.server.ResponseStatusException;
```

```
11
12 @RestController
13 public class ConversionController {
14     @GetMapping("/temperature")
15     public Conversion temperature(@RequestParam double celsius) {
16         validate(celsius);
17         return new Conversion(celsius, "celsius", rounded(celsius
    * 9 / 5 + 32), "fahrenheit");
18     }
19
20     @GetMapping("/distance")
21     public Conversion distance(@RequestParam double kilometres) {
22         validate(kilometres);
23         if (kilometres < 0) {
24             throw new ResponseStatusException(HttpStatus.
    BAD_REQUEST, "Distance cannot be negative");
25         }
26         return new Conversion(kilometres, "kilometres", rounded(
    kilometres * 0.621371), "miles");
27     }
28
29     @GetMapping("/health")
30     public Map<String, String> health() {
31         return Map.of("status", "UP");
32     }
33
34     private static void validate(double value) {
35         if (!Double.isFinite(value) || Math.abs(value) > 1
    _000_000_000) {
36             throw new ResponseStatusException(HttpStatus.
    BAD_REQUEST, "Use a finite value within +/- 1 billion");
37         }
38     }
39
40     private static double rounded(double value) {
41         return BigDecimal.valueOf(value).setScale(4, RoundingMode.
    HALF_UP).doubleValue();
42     }
43
44     public record Conversion(double input, String inputUnit,
    double result, String outputUnit) {}
45 }
```

The Spring Boot entry point and external port setting are shown below.

src/main/java/io/github/shaibaan/converter/ConverterApplication.java

```java
package io.github.shaibaan.converter;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
   SpringBootApplication;

@SpringBootApplication
public class ConverterApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConverterApplication.class, args);
    }
}
```

src/main/resources/application.properties

```
spring.application.name=unit-converter
server.port=${APP_PORT:8080}
server.shutdown=graceful
```

Git Repository and Branching

The repository started empty. Setup was committed on `main`; the API, tests and deployment files were developed on `feature/unit-conversions`. Merge commit `8ed7386` joins the feature branch into main. Both branches are retained on GitHub, and the commits are attributed to Syed Shaibaan Hussain through account `05Shaibaan`.

```
git switch -c feature/unit-conversions
git add src
git commit -m "Add temperature and distance conversions with input
    checks"
git switch main
git merge --no-ff feature/unit-conversions -m "Merge the converter
    API and its DevOps pipeline"
git push -u origin main feature/unit-conversions
```

Maven Build Configuration

The Spring Boot parent supplies compatible dependency and plugin versions. The project targets Java 17 and packages a runnable JAR as `target/unit-converter.jar`. The packaged application was started separately and checked with HTTP requests before deployment.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>
  <groupId>io.github.shaibaan</groupId>
  <artifactId>unit-converter</artifactId>
  <version>1.0.0</version>
  <name>Unit Converter API</name>
  <description>Simple measurement conversions with a complete
    DevOps pipeline</description>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>unit-converter</finalName>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

```
2026-11-13T18:09:31+00:00
Executable JAR started with APP_PORT=8094 java -jar target/unit-
    converter.jar

GET /health
{"status":"UP"}

GET /temperature?celsius=25
```

```
{"input":25.0,"inputUnit":"celsius","result":77.0,"outputUnit":"
    fahrenheit"}

GET /distance?kilometres=5
{"input":5.0,"inputUnit":"kilometres","result":3.1069,"outputUnit
    ":"miles"}

GET /distance?kilometres=-1 (invalid input)
HTTP 400
```

Jenkins Pipeline

The `unit-converter` job uses Pipeline script from SCM, Git, the repository URL above, branch `*/main`, and script path `Jenkinsfile`. The first run loads the SCM polling trigger. Later source changes are detected every minute. The pipeline tests first, packages the application only after tests pass, publishes JUnit results, and archives the JAR with a fingerprint.

`Jenkinsfile`

```
pipeline {
    agent any
    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }
    triggers { pollSCM('* * * * *') }
    stages {
        stage('Test') {
            steps {
                script {
                    if (isUnix()) { sh 'mvn -B clean test' }
                    else { bat 'mvn -B clean test' }
                }
            }
            post { always { junit 'target/surefire-reports/*.xml'
    } }
        }
        stage('Build') {
            steps {
                script {
                    if (isUnix()) { sh 'mvn -B -DskipTests package
    ' }
                    else { bat 'mvn -B -DskipTests package' }
                }
                archiveArtifacts artifacts: 'target/unit-converter
    .jar', fingerprint: true
            }
        }
```

```
    }
}
```

Enhancement: Automated Tests Before Packaging

Automated testing was chosen because a converter can return a successful HTTP response while still calculating the wrong number. The tests check known conversion results, negative temperature, zero distance, invalid values and health. Parameterized cases also cover NaN, infinity, overflow, malformed text, negative distance and the upper input limit. Fifteen test cases run in total. These tests exercise the controller through Spring MockMvc without starting a network server; separate HTTP checks verify the packaged application.

`src/test/java/io/github/shaibaan/converter/ConversionControllerTest.java`

```
package io.github.shaibaan.converter;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import static org.springframework.test.web.servlet.request.
   MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.
   MockMvcResultMatchers.*;

class ConversionControllerTest {
    private MockMvc mvc;

    @BeforeEach
    void setUp() {
        mvc = MockMvcBuilders.standaloneSetup(new
   ConversionController()).build();
    }

    @Test
    void convertsBoilingWaterToFahrenheit() throws Exception {
        mvc.perform(get("/temperature").param("celsius", "100"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.result").value(212.0))
                .andExpect(jsonPath("$.outputUnit").value("
   fahrenheit"));
    }

    @Test
    void convertsNegativeTemperature() throws Exception {
```

```
31          mvc.perform(get("/temperature").param("celsius", "-40"))
32                  .andExpect(status().isOk()).andExpect(jsonPath("$.
    result").value(-40.0));
33      }
34
35      @Test
36      void convertsFiveKilometresToMiles() throws Exception {
37          mvc.perform(get("/distance").param("kilometres", "5"))
38                  .andExpect(status().isOk())
39                  .andExpect(jsonPath("$.result").value(3.1069))
40                  .andExpect(jsonPath("$.inputUnit").value("
    kilometres"));
41      }
42
43      @Test
44      void handlesZeroDistance() throws Exception {
45          mvc.perform(get("/distance").param("kilometres", "0"))
46                  .andExpect(status().isOk()).andExpect(jsonPath("$.
    result").value(0.0));
47      }
48
49      @ParameterizedTest
50      @ValueSource(strings = {"NaN", "Infinity", "1e309", "
    1000000001", "abc"})
51      void rejectsInvalidTemperature(String value) throws Exception
    {
52          mvc.perform(get("/temperature").param("celsius", value)).
    andExpect(status().isBadRequest());
53      }
54
55      @ParameterizedTest
56      @ValueSource(strings = {"-1", "NaN", "-Infinity", "1000000001"
    })
57      void rejectsInvalidDistance(String value) throws Exception {
58          mvc.perform(get("/distance").param("kilometres", value)).
    andExpect(status().isBadRequest());
59      }
60
61      @Test
62      void requiresTheInputParameter() throws Exception {
63          mvc.perform(get("/temperature")).andExpect(status().
    isBadRequest());
64          mvc.perform(get("/distance")).andExpect(status().
    isBadRequest());
65      }
66
67      @Test
68      void returnsHealthStatus() throws Exception {
```

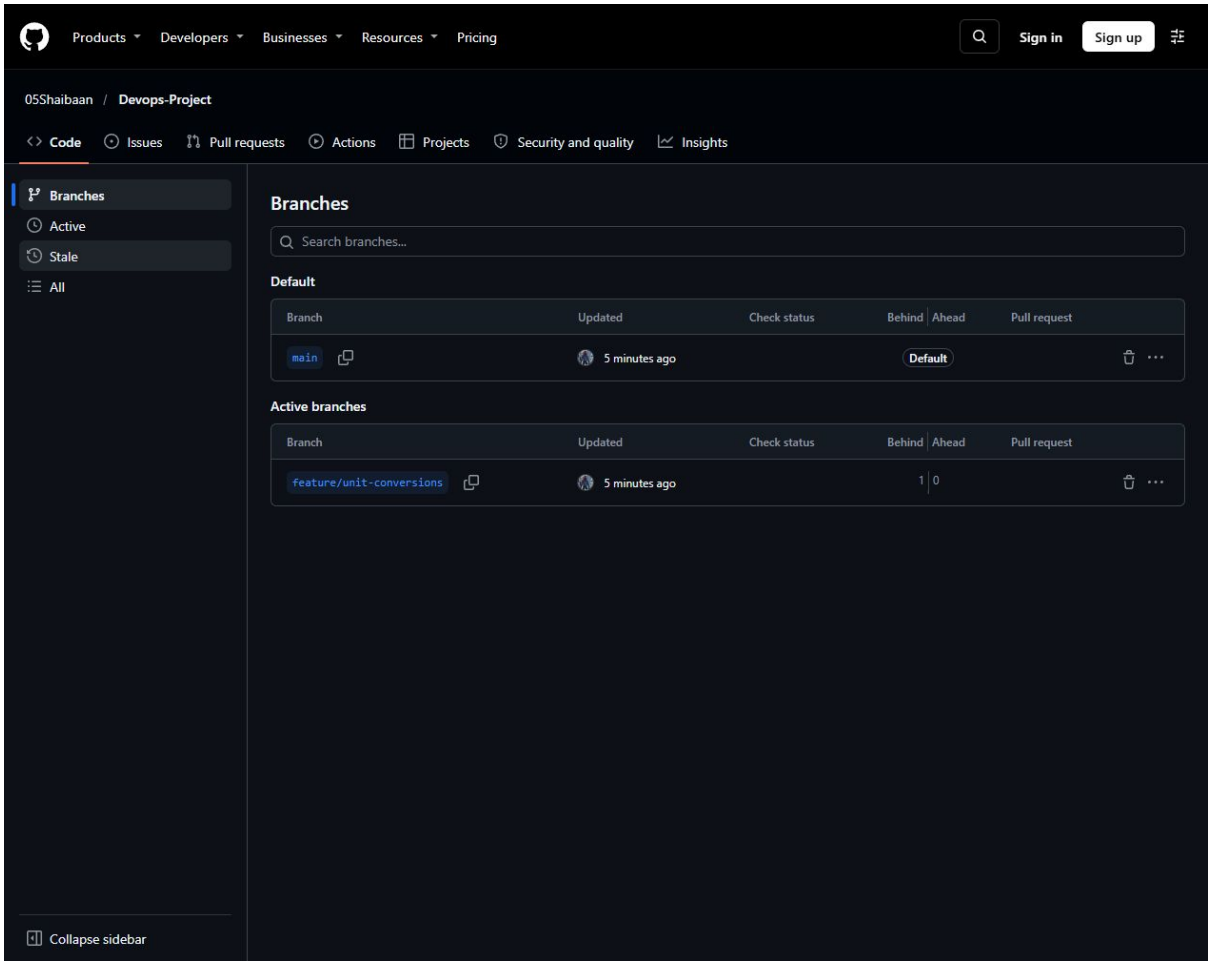
```
69          mvc.perform(get("/health")).andExpect(status().isOk())
70                  .andExpect(jsonPath("$.status").value("UP"));
71      }
72  }
```

Description of the Activity

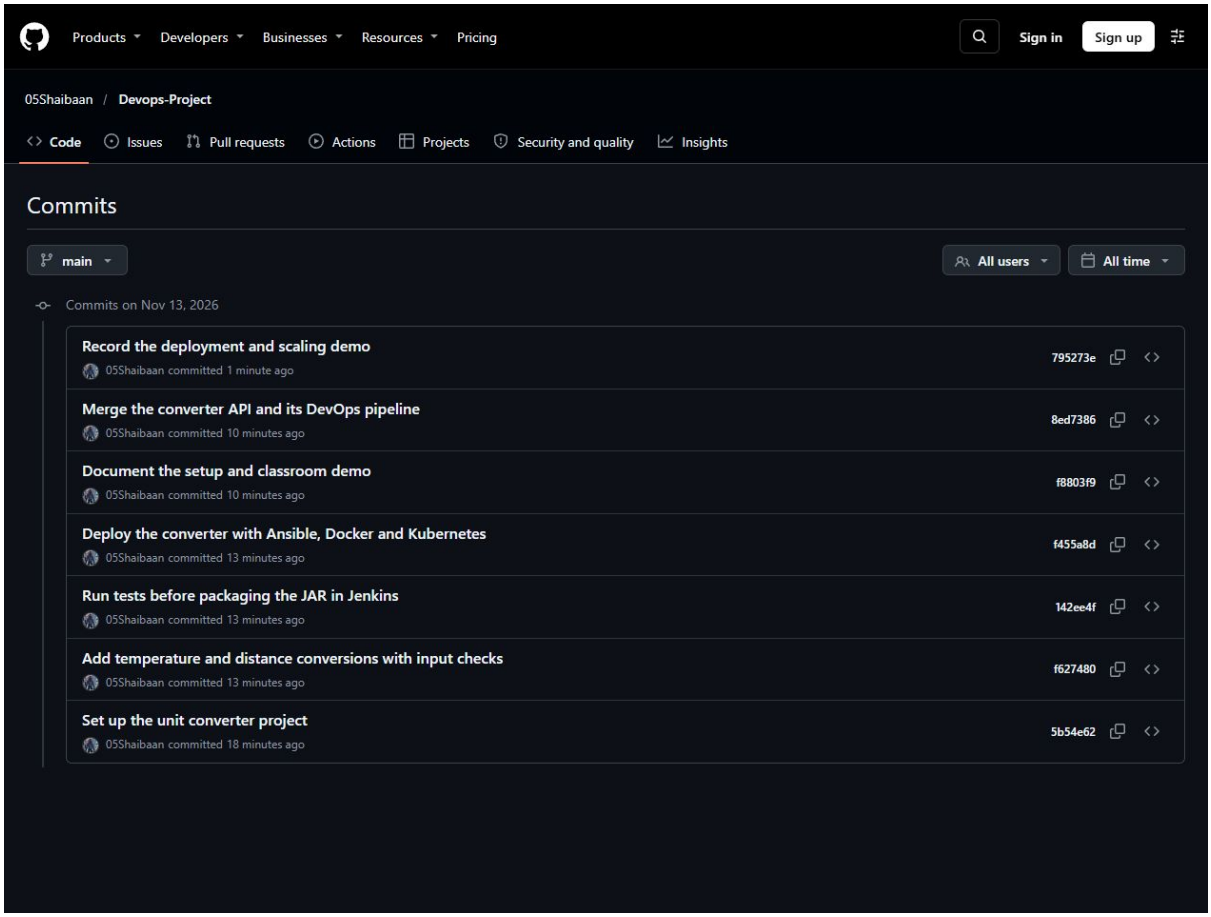
The conversion checks were written before the controller implementation. After implementing the routes, the Maven build passed and the JAR returned 77 Fahrenheit for 25 Celsius and 3.1069 miles for 5 kilometres. The feature branch was merged, Jenkins was configured from GitHub, and a later evidence commit was pushed without pressing Build Now again. SCM polling detected that change and ran the pipeline automatically.

Results

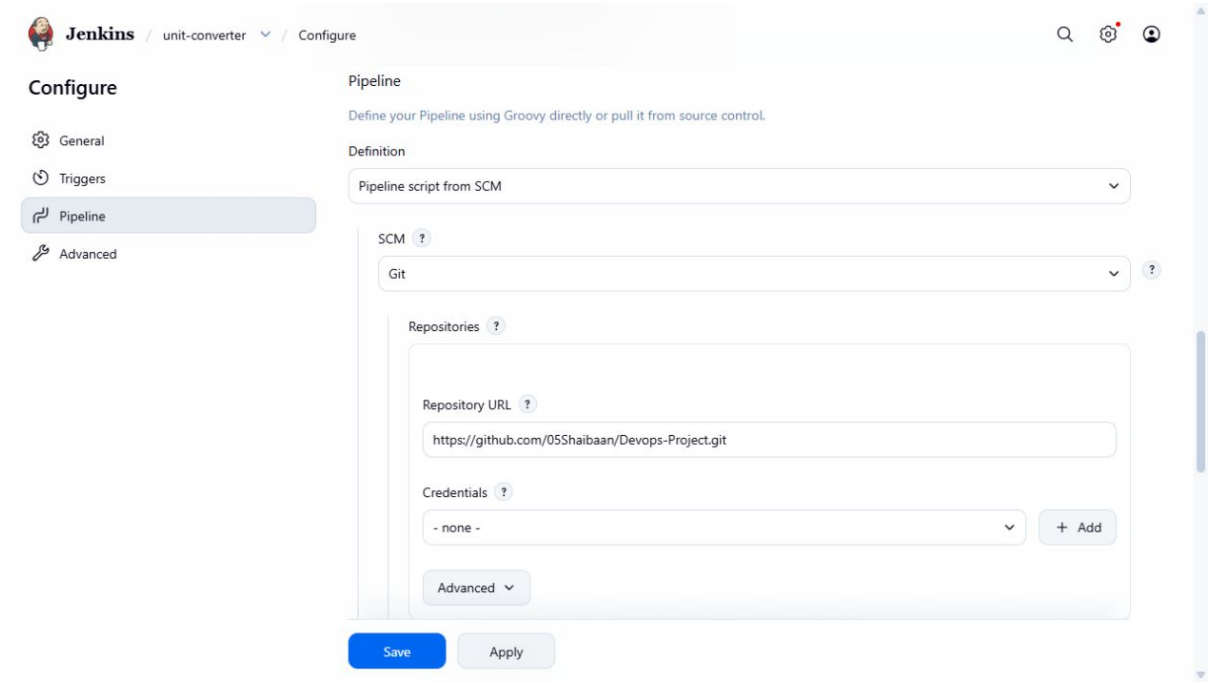
Jenkins build #2 completed with `SUCCESS` after an SCM change. All 15 tests passed, with zero failures and zero skipped cases. The build archived `target/unit-converter.jar`. Screenshots below show the repository, configuration, successful execution and test results. Full console output is retained in `evidences/logs/jenkins-console.txt`.



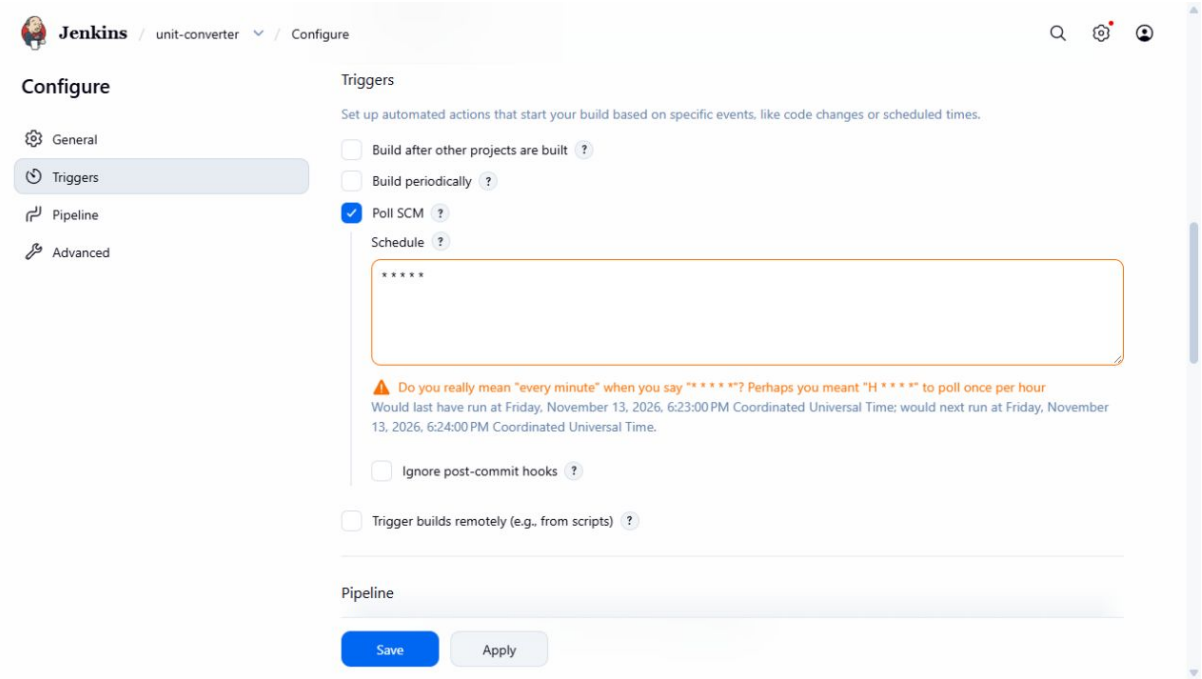
The main and feature/unit-conversions branches on GitHub.



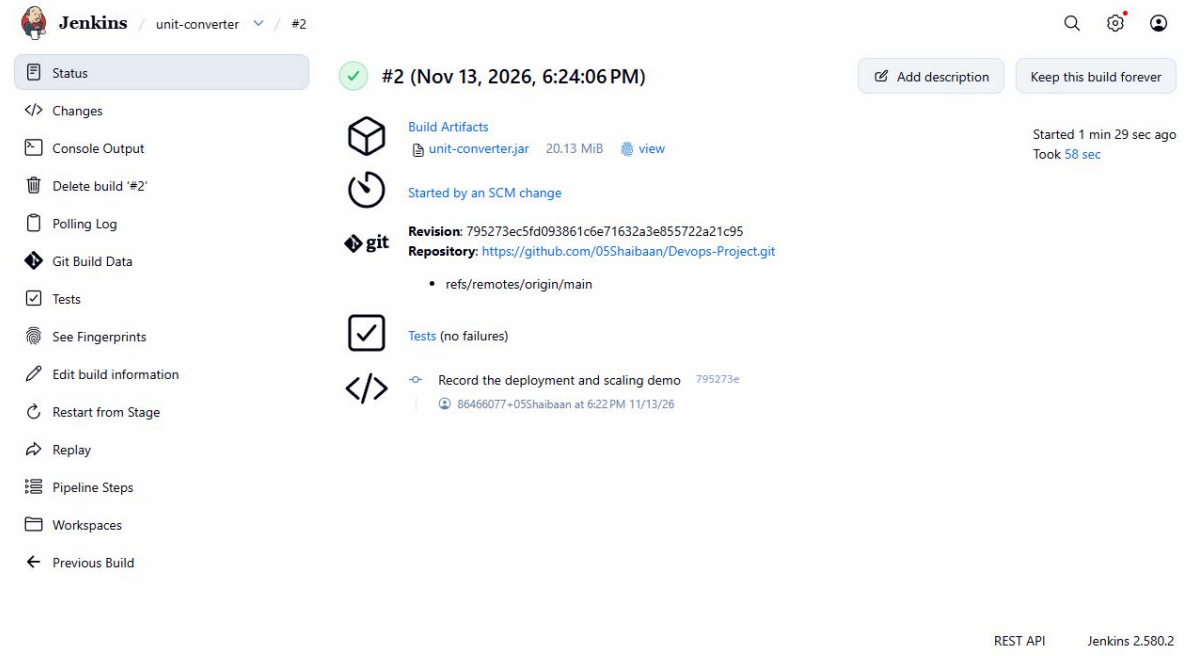
Commit history includes the feature merge and Shaibaan attribution.



Jenkins reads this repository and the Jenkinsfile from SCM.



SCM polling checks for new commits every minute.



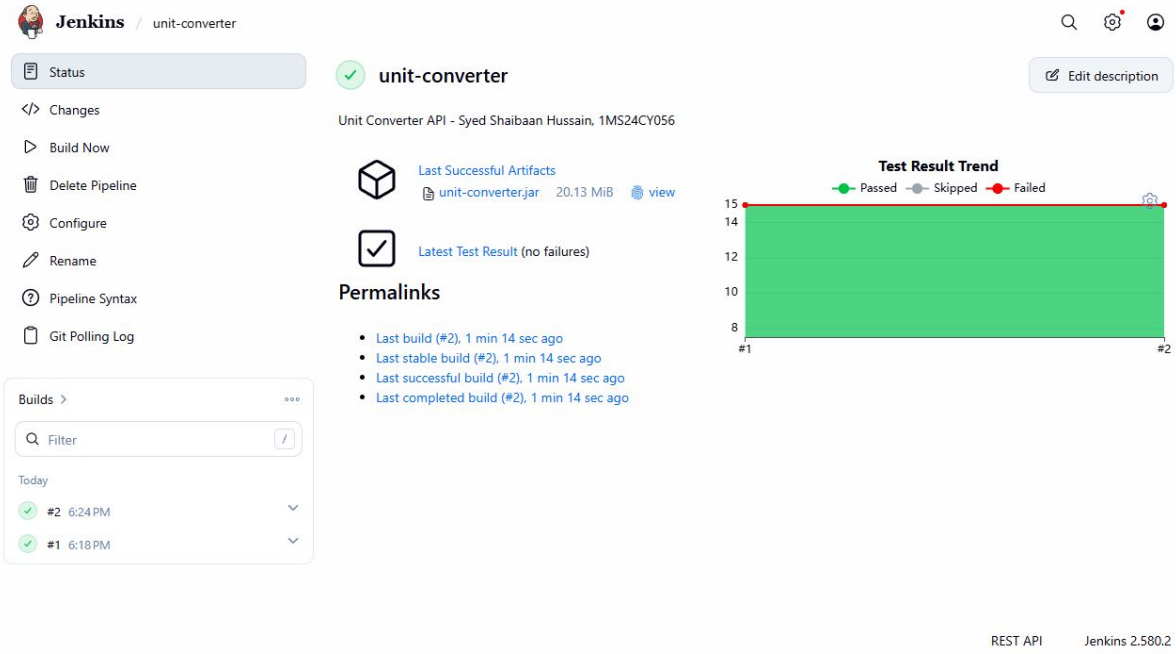
Successful pipeline triggered automatically by an SCM change.

06. Jenkins Test and Build console output

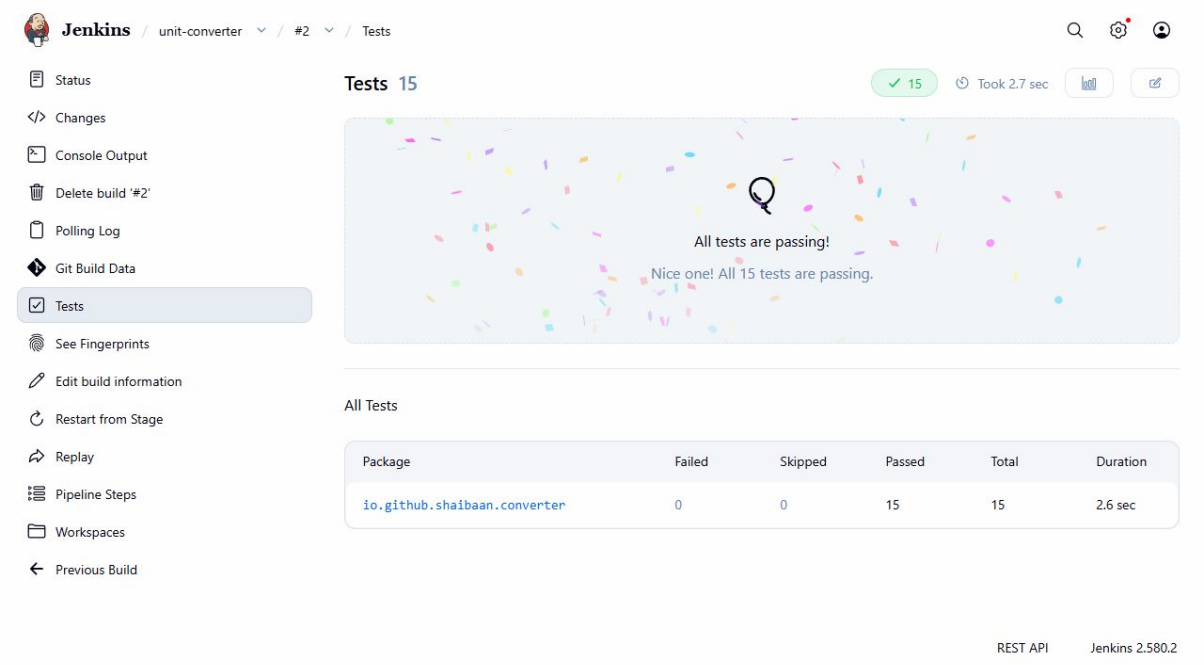
Unit Converter API | Syed Shaibaan Hussain | 1MS24CY056
Selected output from the actual SCM-triggered Jenkins build #2.
Full source: evidences/logs/jenkins-console.txt

```
Started by an SCM change
Obtained Jenkinsfile from git https://github.com/05Shaibaan/Devops-Project.git
[Pipeline] { (Test)
[2026-11-13T18:24:10.762Z] + mvn -B clean test
[2026-11-13T18:24:40.612Z] [INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.788 s -- in
io.github.shaibaan.converter.ConversionControllerTest
[2026-11-13T18:24:40.863Z] [INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[2026-11-13T18:24:40.863Z] [INFO] BUILD SUCCESS
[Pipeline] { (Build)
[2026-11-13T18:24:41.594Z] + mvn -B -DskipTests package
[2026-11-13T18:25:05.169Z] [INFO] BUILD SUCCESS
[2026-11-13T18:25:05.289Z] Archiving artifacts
[2026-11-13T18:25:05.309Z] Recording fingerprints
Finished: SUCCESS
```

Console output confirms the tests and packaging completed.



Jenkins retains the executable JAR as an archived artifact.



All fifteen automated tests passed.

Conclusion

This component produced a working converter and a repeatable build-and-test pipeline. The branch history records the implementation, the automatic SCM build demonstrates CI, and the archived JAR supplies the exact application needed by the next component. Test failures stop packaging and make calculation or validation regressions visible early.

Resume line. Built a Java Unit Converter API and automated its Maven build in Jenkins with SCM polling, fifteen tests and an archived executable JAR.

References

1. Spring Boot servlet web applications: `https://docs.spring.io/spring-boot/3.5/reference/web/servlet.html`
2. Maven build lifecycle: `https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html`
3. Jenkins Pipeline syntax: `https://www.jenkins.io/doc/book/pipeline/syntax/`
4. Git branching and merging: `https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging`

Component 2: Configuration Management and Kubernetes Deployment

Abstract

This component deploys the same Unit Converter API through Ansible, Docker and Kubernetes. Ansible configures an Ubuntu service; Docker packages the built JAR; Kubernetes exposes it through a Service and demonstrates three, five and three replicas. A ConfigMap supplies the application port, while health probes control readiness and detect an unhealthy container.

Introduction

Configuration management records server setup in a playbook so it can be repeated. A Docker image then packages the runtime and application together. Kubernetes manages copies of that image, replaces failed containers and gives clients a stable Service address. Keeping the API stateless means every replica performs the same calculation without shared storage.

Implementation

Development Environment

The deployment uses Ubuntu under WSL2, Ansible Core 2.20.1, Docker Engine 29.1.3, Kind 0.31.0 and kubectl 1.35.0. The dedicated cluster is `converter-demo`; its namespace is also `converter-demo`. The Ansible service uses port 8093, the container test uses 8095, and the Kubernetes Service is forwarded to 8096, keeping the three demonstrations independent.

Ansible Inventory, Playbook and Role

The inventory targets the local Ubuntu host. The `converter` role was initialized using `ansible-galaxy role init converter`. It installs Java 17, creates an unprivileged service account, copies the built JAR, renders the systemd unit and checks the health route. Variables define the port, user, directory and artifact path. Changed files notify a restart handler; unchanged files do not restart the application.

`ansible/inventory`

```
1 [app_servers]
2 localhost ansible_connection=local
```

`ansible/playbook.yml`

```
1 ---
2 - name: Configure the Unit Converter API server
3   hosts: app_servers
4   become: true
5   roles:
6     - converter
```

ansible/roles/converter/defaults/main.yml

```yaml
---
app_port: 8093
app_directory: /opt/unit-converter
app_user: unit-converter
app_jar: "{{ playbook_dir }}/../target/unit-converter.jar"
```

ansible/roles/converter/tasks/main.yml

```yaml
---
- name: Install the Java runtime
  ansible.builtin.apt:
    name: openjdk-17-jre-headless
    state: present
    update_cache: true
    cache_valid_time: 3600

- name: Create a service account
  ansible.builtin.user:
    name: "{{ app_user }}"
    system: true
    shell: /usr/sbin/nologin
    create_home: false

- name: Create the application directory
  ansible.builtin.file:
    path: "{{ app_directory }}"
    state: directory
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: '0755'

- name: Copy the built application
  ansible.builtin.copy:
    src: "{{ app_jar }}"
    dest: "{{ app_directory }}/unit-converter.jar"
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: '0644'
  notify: Restart unit converter

- name: Install the service definition
  ansible.builtin.template:
    src: unit-converter.service.j2
    dest: /etc/systemd/system/unit-converter.service
    mode: '0644'
  notify: Restart unit converter
```

```
39
40 - name: Apply pending service changes
41   ansible.builtin.meta: flush_handlers
42
43 - name: Enable and start the service
44   ansible.builtin.systemd_service:
45     name: unit-converter
46     enabled: true
47     state: started
48     daemon_reload: true
49
50 - name: Wait for the API to become healthy
51   ansible.builtin.uri:
52     url: "http://127.0.0.1:{{ app_port }}/health"
53     return_content: true
54   register: health
55   until: health.status == 200 and health.json.status == 'UP'
56   retries: 20
57   delay: 3
```

ansible/roles/converter/handlers/main.yml

```
1 ---
2 - name: Restart unit converter
3   ansible.builtin.systemd_service:
4     name: unit-converter
5     state: restarted
6     daemon_reload: true
```

ansible/roles/converter/templates/unit-converter.service.j2

```
1 [Unit]
2 Description=Unit Converter API
3 After=network.target
4
5 [Service]
6 User={{ app_user }}
7 Group={{ app_user }}
8 WorkingDirectory={{ app_directory }}
9 Environment=APP_PORT={{ app_port }}
10 ExecStart=/usr/bin/java -jar {{ app_directory }}/unit-converter.
    jar
11 Restart=on-failure
12 RestartSec=5
13 NoNewPrivileges=true
14 PrivateTmp=true
15 ProtectSystem=strict
16 ProtectHome=true
```

```
TimeoutStopSec=20

[Install]
WantedBy=multi-user.target
```

Docker Image and Local Container

The image uses the Java 17 runtime, copies the executable JAR and starts it as user 10001. The build context contains only the packaged artifact. A local container was started with a loopback port mapping and checked with the distance route before the image was loaded into Kind.

`Dockerfile`

```
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --chown=10001:10001 target/unit-converter.jar app.jar
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar
    "]
```

`.dockerignore`

```
**
!target/
!target/unit-converter.jar
```

Kubernetes Deployment and Service

The Deployment requests three replicas of `unit-converter:1.0`. Its pod label and Service selector both use `app: unit-converter`. The ClusterIP Service maps port 8080 to the named HTTP container port. Resource requests and limits keep the local demonstration bounded. The namespace and ConfigMap are created before the application is started.

`k8s/namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: converter-demo
```

`k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: converter-demo
  name: unit-converter
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: unit-converter
  template:
    metadata:
      labels:
        app: unit-converter
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
      containers:
        - name: unit-converter
          image: unit-converter:1.0
          imagePullPolicy: IfNotPresent
          envFrom:
            - configMapRef:
                name: unit-converter-config
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 192Mi
            limits:
              cpu: 500m
              memory: 384Mi
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
          startupProbe:
            httpGet:
              path: /health
              port: http
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet:
              path: /health
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
```

```
              path: /health
              port: http
            periodSeconds: 10
```

k8s/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  namespace: converter-demo
  name: unit-converter
spec:
  selector:
    app: unit-converter
  ports:
    - name: http
      port: 8080
      targetPort: http
  type: ClusterIP
```

scripts/deploy.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")/.."

# Use the dedicated demo cluster so another Kubernetes context is never changed.
kind load docker-image unit-converter:1.0 --name converter-demo
kubectl --context kind-converter-demo apply -f k8s/namespace.yaml
kubectl --context kind-converter-demo -n converter-demo apply -f k8s/
kubectl --context kind-converter-demo -n converter-demo rollout status deployment/unit-converter --timeout=180s
kubectl --context kind-converter-demo -n converter-demo get pods -l app=unit-converter
kubectl --context kind-converter-demo -n converter-demo get service unit-converter
```

Scaling

The application starts with three ready replicas. Scaling to five adds two pods; scaling back to three removes them. The script waits for rollout completion and the reported replica count at each step. This demonstrates manual horizontal scaling; it does not claim an autoscaler was installed.

scripts/scale-demo.sh

```
#!/usr/bin/env bash
set -euo pipefail

for replicas in 3 5 3; do
    echo "Scaling unit-converter to $replicas replicas"
    kubectl --context kind-converter-demo -n converter-demo scale deployment/unit-converter --replicas="$replicas"
    kubectl --context kind-converter-demo -n converter-demo rollout status deployment/unit-converter --timeout=180s
    kubectl --context kind-converter-demo -n converter-demo wait deployment/unit-converter \
        --for="jsonpath={.status.replicas}=$replicas" --timeout=180s
    kubectl --context kind-converter-demo -n converter-demo get pods -l app=unit-converter
done
```

Enhancement: ConfigMap and Health Probes

The ConfigMap supplies `APP_PORT=8080` through the Deployment's `envFrom`. This separates an application setting from the image. The startup probe allows time for Java to start, readiness keeps traffic away until the API responds, and liveness allows Kubernetes to restart an unhealthy container. All three use `/health`. Changing environment settings requires restarting the pods; changing the listening port also requires matching the container port and probes.

k8s/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: converter-demo
  name: unit-converter-config
data:
  APP_PORT: "8080"
```

Description of the Activity

The Ansible playbook first installed the service and successfully checked its response. A second unchanged run confirmed idempotence. The same JAR was built into a Docker image and tested locally. That image was loaded into a separate Kind cluster, the manifests were applied, and the Service was checked with both conversion routes. Replicas were increased to five and returned to three. Finally, the container environment was inspected to confirm that the ConfigMap value reached the running application.

Results

Ansible completed with zero unreachable or failed hosts; the repeated run reported zero changes. Docker built the image and returned 3.1069 miles for 5 kilometres. The Kind node became Ready, and Kubernetes showed three, five and three ready pods. Service requests returned 77 Fahrenheit and 3.1069 miles. The running container reported `APP_PORT=8080`. Screenshots 09–21 display actual recorded command output; the complete logs are included in the repository.

09. Ansible inventory and playbook files

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/09-ansible-inventory-playbook.txt

```
2026-11-13T23:40:28+05:30
ansible-galaxy role init converter
Role initialized before configuration.

Inventory:
[app_servers]
localhost ansible_connection=local

Playbook:
---
- name: Configure the Unit Converter API server
  hosts: app_servers
  become: true
  roles:
    - converter

Role files:
ansible/roles/converter/defaults/main.yml
ansible/roles/converter/handlers/main.yml
ansible/roles/converter/tasks/main.yml
ansible/roles/converter/templates/unit-converter.service.j2
```

Inventory, playbook and initialized role files.

10. Successful Ansible run and PLAY RECAP

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Selected recorded output; full log retained. Source: evidences/logs/10-ansible-play-recap.txt

```
2026-11-13T23:40:29+05:30

Selected output from: ansible-playbook -i inventory playbook.yml

Initial run:
RUNNING HANDLER [converter : Restart unit converter]
PLAY RECAP *********************************************************************
localhost                  : ok=9    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

Second unchanged run:
PLAY RECAP *********************************************************************
localhost                  : ok=8    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

Service response:
{"input":25.0,"inputUnit":"celsius","result":77.0,"outputUnit":"fahrenheit"}
```

Successful Ansible execution and unchanged second run.

11. Dockerfile used by the Unit Converter API

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/11-dockerfile.txt

```
2026-11-13T23:41:03+05:30
Command: cat Dockerfile

FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --chown=10001:10001 target/unit-converter.jar app.jar
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Dockerfile used to package the JAR.

12. Successful Docker image build

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/12-docker-image-build.txt

```
2026-11-13T23:41:03+05:30
Command: /usr/bin/docker build -t unit-converter:1.0 .

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  21.18MB

Step 1/6 : FROM eclipse-temurin:17-jre-jammy
 ---> 2a9eb226abb6
Step 2/6 : WORKDIR /app
 ---> Using cache
 ---> 2a3f4b699abd
Step 3/6 : COPY --chown=10001:10001 target/unit-converter.jar app.jar
 ---> 51d0e76f5421
Step 4/6 : USER 10001:10001
 ---> Running in b5d3ebf5519c
 ---> Removed intermediate container b5d3ebf5519c
 ---> dc6daa766931
Step 5/6 : EXPOSE 8080
 ---> Running in 36db520eb901
 ---> Removed intermediate container 36db520eb901
 ---> 51fda316be25
Step 6/6 : ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
 ---> Running in 4c3d4dc67a4d
 ---> Removed intermediate container 4c3d4dc67a4d
 ---> 7a25a0f92182
Successfully built 7a25a0f92182
Successfully tagged unit-converter:1.0
```

Docker image build completed successfully.

13. Docker images command output

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/13-docker-images.txt

```
2026-11-13T23:41:09+05:30
Command: /usr/bin/docker images unit-converter:1.0

WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
unit-converter:1.0   7a25a0f92182       415MB          113MB
```

Built unit-converter image in the local image list.

14. Running Docker container and health response

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/14-running-docker-container.txt

```
2026-11-13T23:41:14+05:30
Command: docker ps --filter name=converter-evidence

CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS         PORTS                     NAMES
48bb5b86e4b7   unit-converter:1.0   "java -XX:MaxRAMPerc…"   4 seconds ago   Up 3 seconds   127.0.0.1:8095->8080/tcp   converter-
evidence

GET /distance?kilometres=5
{"input":5.0,"inputUnit":"kilometres","result":3.1069,"outputUnit":"miles"}
```

Running container returns the distance conversion.

15. Kind Kubernetes node showing Ready

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/15-kind-node-ready.txt

```
2026-11-13T23:44:07+05:30
Command: kubectl --context kind-converter-demo get nodes -o wide

NAME                           STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                     CONTAINER-RUNTIME
converter-demo-control-plane   Ready    control-plane   63s   v1.35.0   172.18.0.3    <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   containerd://2.2.0
```

The dedicated Kind control-plane node is Ready.

16. Three ready Kubernetes replicas

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/16-kubernetes-three-replicas.txt

```
2026-11-13T23:44:08+05:30
Command: kubectl --context kind-converter-demo -n converter-demo get pods -l app=unit-converter

NAME                              READY   STATUS    RESTARTS   AGE
unit-converter-bb776f9c6-f8vqv   1/1     Running   0          21s
unit-converter-bb776f9c6-fqzqt   1/1     Running   0          21s
unit-converter-bb776f9c6-gp7wd   1/1     Running   0          21s
```

Three initial replicas are ready.

17. Running Unit Converter API Kubernetes Service

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/17-kubernetes-service.txt

```
2026-11-13T23:44:09+05:30
Command: kubectl --context kind-converter-demo -n converter-demo get svc unit-converter

NAME             TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
unit-converter   ClusterIP   10.96.94.188   <none>        8080/TCP   21s
```

The ClusterIP Service exposes the converter.

18. Application responding through the Service

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/18-application-through-service.txt

```
2026-11-13T23:44:10+05:30
Command: kubectl --context kind-converter-demo -n converter-demo port-forward service/unit-converter 8096:8080

GET /health
{"status":"UP"}

GET /temperature?celsius=25
{"input":25.0,"inputUnit":"celsius","result":77.0,"outputUnit":"fahrenheit"}

GET /distance?kilometres=5
{"input":5.0,"inputUnit":"kilometres","result":3.1069,"outputUnit":"miles"}
```

Application responses through the Kubernetes Service.

19. Five ready replicas after scaling up

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/19-kubernetes-five-replicas.txt

```
2026-11-13T23:44:10+05:30
Command: kubectl scale deployment/unit-converter --replicas=5

deployment.apps/unit-converter scaled
Waiting for deployment "unit-converter" rollout to finish: 3 of 5 updated replicas are available...
Waiting for deployment "unit-converter" rollout to finish: 4 of 5 updated replicas are available...
deployment "unit-converter" successfully rolled out
deployment.apps/unit-converter condition met
pod/unit-converter-bb776f9c6-68hqq condition met
pod/unit-converter-bb776f9c6-f8vqv condition met
pod/unit-converter-bb776f9c6-fqzqt condition met
pod/unit-converter-bb776f9c6-gp7wd condition met
pod/unit-converter-bb776f9c6-gt6r7 condition met
NAME                              READY   STATUS    RESTARTS   AGE
unit-converter-bb776f9c6-68hqq   1/1     Running   0          17s
unit-converter-bb776f9c6-f8vqv   1/1     Running   0          40s
unit-converter-bb776f9c6-fqzqt   1/1     Running   0          40s
unit-converter-bb776f9c6-gp7wd   1/1     Running   0          40s
unit-converter-bb776f9c6-gt6r7   1/1     Running   0          17s
```

Five ready replicas after scaling up.

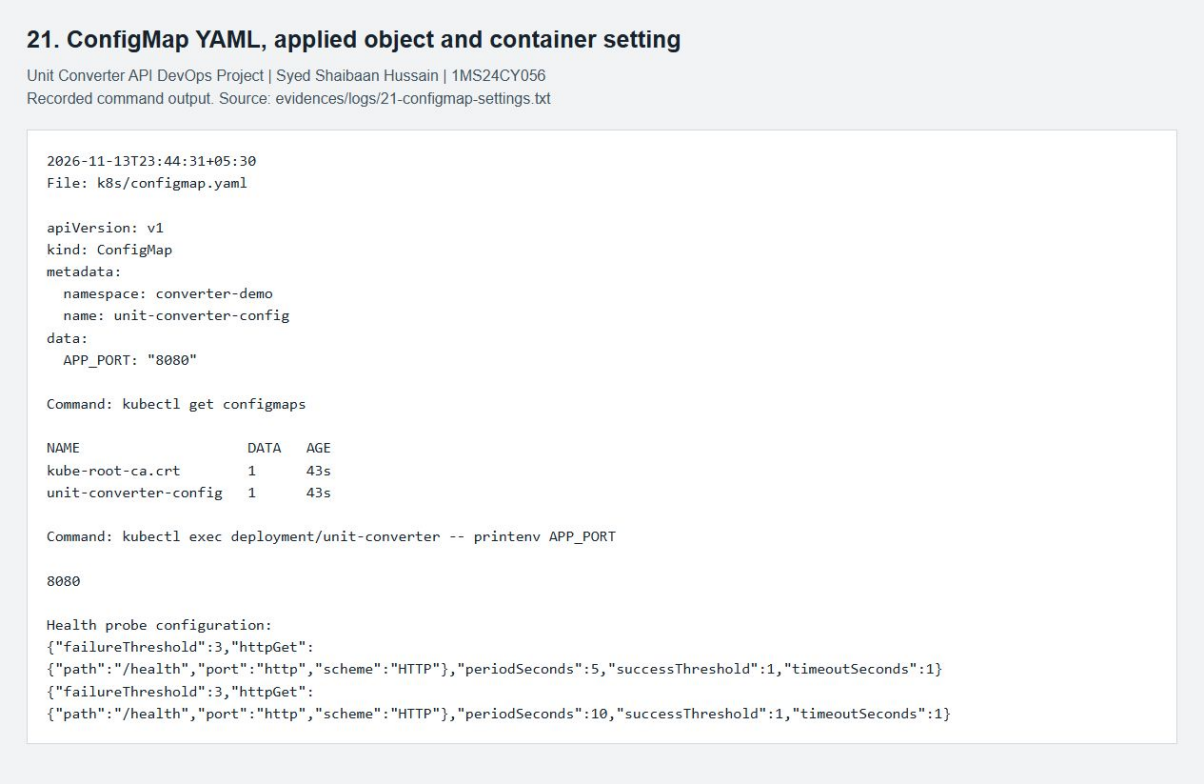
20. Three ready replicas after scaling down

Unit Converter API DevOps Project | Syed Shaibaan Hussain | 1MS24CY056
Recorded command output. Source: evidences/logs/20-kubernetes-scaled-down.txt

```
2026-11-13T23:44:28+05:30
Command: kubectl scale deployment/unit-converter --replicas=3

deployment.apps/unit-converter scaled
deployment "unit-converter" successfully rolled out
deployment.apps/unit-converter condition met
pod/unit-converter-bb776f9c6-f8vqv condition met
pod/unit-converter-bb776f9c6-fqzqt condition met
pod/unit-converter-bb776f9c6-gp7wd condition met
NAME                              READY   STATUS    RESTARTS   AGE
unit-converter-bb776f9c6-f8vqv   1/1     Running   0          42s
unit-converter-bb776f9c6-fqzqt   1/1     Running   0          42s
unit-converter-bb776f9c6-gp7wd   1/1     Running   0          42s
```

Three ready replicas after scaling down.



ConfigMap value in the container and health probe configuration.

Conclusion

The deployment component demonstrated repeatable server setup, a working container image and a Kubernetes Service with controlled scaling. The ConfigMap and probes make configuration and application health visible. The result remains a small local demonstration with no database, paid cloud service or additional microservice.

Resume line. Configured an Ubuntu service with an Ansible role and restart handler, then deployed its Docker image on Kubernetes with 3–5–3 scaling, a ConfigMap and health probes.

References

1. Ansible roles: `https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html`
2. Docker build concepts: `https://docs.docker.com/build/concepts/dockerfile/`
3. Kind quick start: `https://kind.sigs.k8s.io/docs/user/quick-start/`
4. Kubernetes Deployments: `https://kubernetes.io/docs/concepts/workloads/controllers/deployment/`
5. ConfigMaps: `https://kubernetes.io/docs/concepts/configuration/configmap/`
6. Health probes: `https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/`

Submission details. Repository: `https://github.com/05Shaibaan/Devops-Project`. Student: Syed Shaibaan Hussain, USN 1MS24CY056. Candidate and faculty signatures are left for the appropriate signatories, and assessment marks are left for evaluation. The coordinator must be added as a collaborator once their GitHub username is available. The course code follows the assignment brief, 25CYAEC59; the supplied template used 24CYAEC59, so the coordinator should confirm the applicable code before submission.